QuizMe: Project Documentation

Project link: https://github.com/Ghaby-X/Amalitech/tree/lab-agile-devops/agile-devops-lab

This document consolidates the agile/DevOps evidence for the QuizMe project: backlog and sprint plans, codebase delivery history, CI/CD setup, testing evidence, sprint reviews, and retrospectives.

1. Backlog & Sprint Plans

Product Vision

To make learning fun, accessible, and engaging by providing a simple flashcard quiz app that helps people expand their general knowledge and cultural awareness through quick, interactive quizzes they can enjoy anytime.

Product Backlog (User Stories)

#	User Story	Story Points	Priority
1	As a user, I want to start a quiz with one tap so I can begin answering questions quickly	SP-1	100
2	As a user, I want to be able to answer either multiple choice or True/False questions so that I can test my knowledge	SP-3	100
3	As a user, I want to be able to get instant feedback so that I can know whether my answers are correct or not	SP-3	100
4	As a user, I want to be able to move to the next question so that I can proceed with the quiz	SP-2	100
5	As a user, I want to be able to see the scores of my quiz	SP-2	80

#	User Story	Story Points	Priority
	as I progress through each of them		
6	As a user, I want to be able to choose the category of quiz so that I get questions related to only that category	SP-5	40
7	As a user, I want to be able to finish a quiz so that I don't end up in a continuous stream of question answering	SP-2	70
8	As a user, I want to be able to see my general performance and categorical performance after I finish a quiz so that I can know which categories I am more strongly suited for	SP-3	60

Full acceptance criteria for each story are in sprint-docs/001_Vision_And_Backlog.md.

Definition of Done (DoD)

A user story is considered done when all of the following are true:

- All acceptance criteria for the story are met
- The feature renders correctly in the browser without console errors
- At least one unit test covers the core logic of the story
- Code is committed with a meaningful commit message
- The CI pipeline passes (build + tests)
- The feature is usable without breaking any previously delivered story

Sprint Breakdown

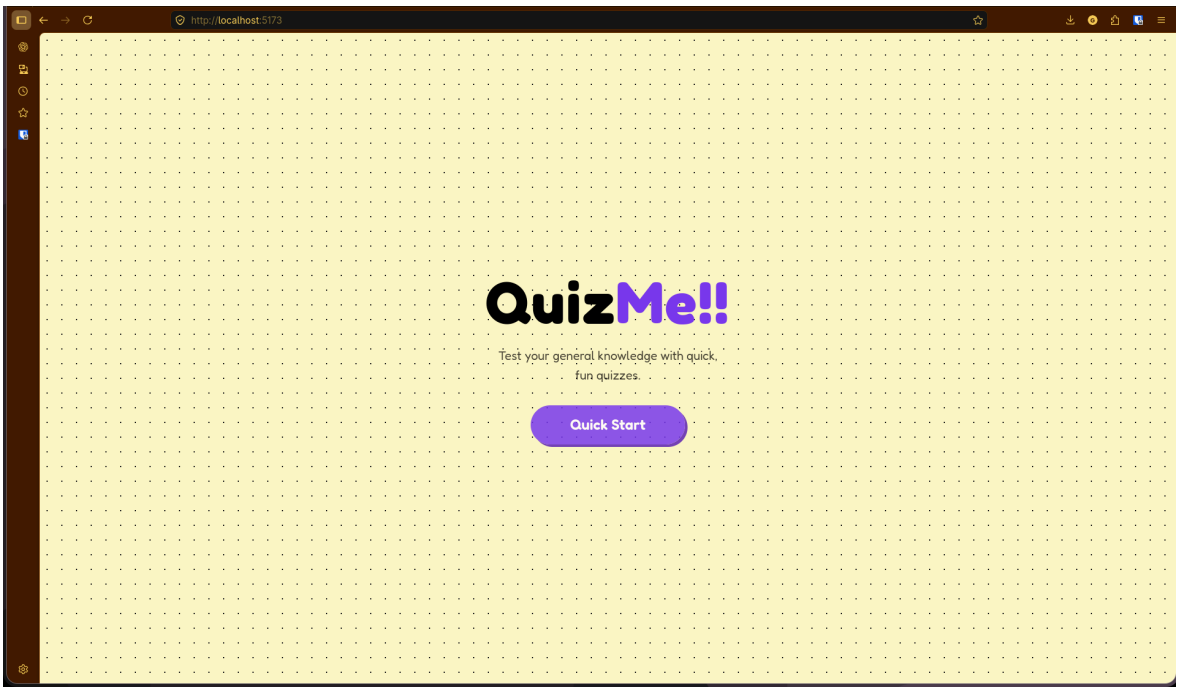
Sprint 1: Core Quiz Flow (7 SP)

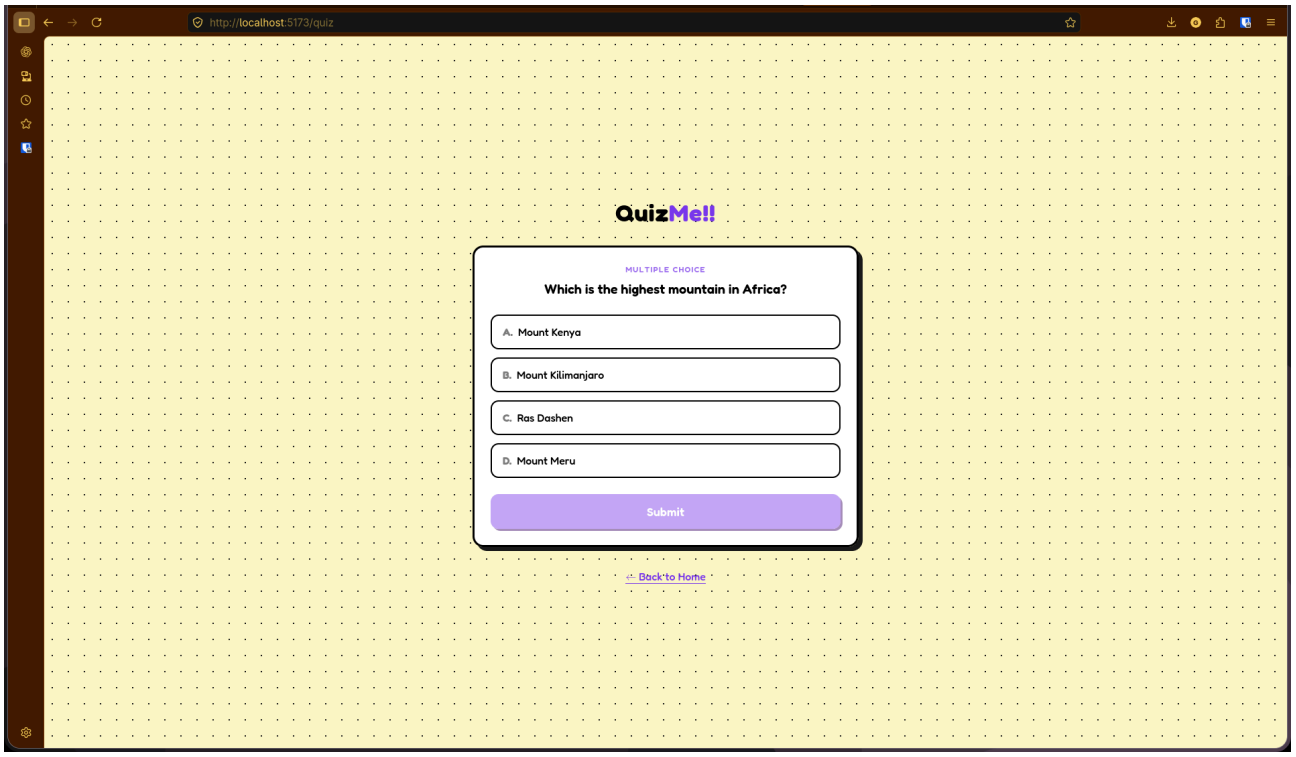
Goal: Deliver a working quiz flow where a user can start a quiz, answer multiple choice and true/false questions, and receive instant feedback.

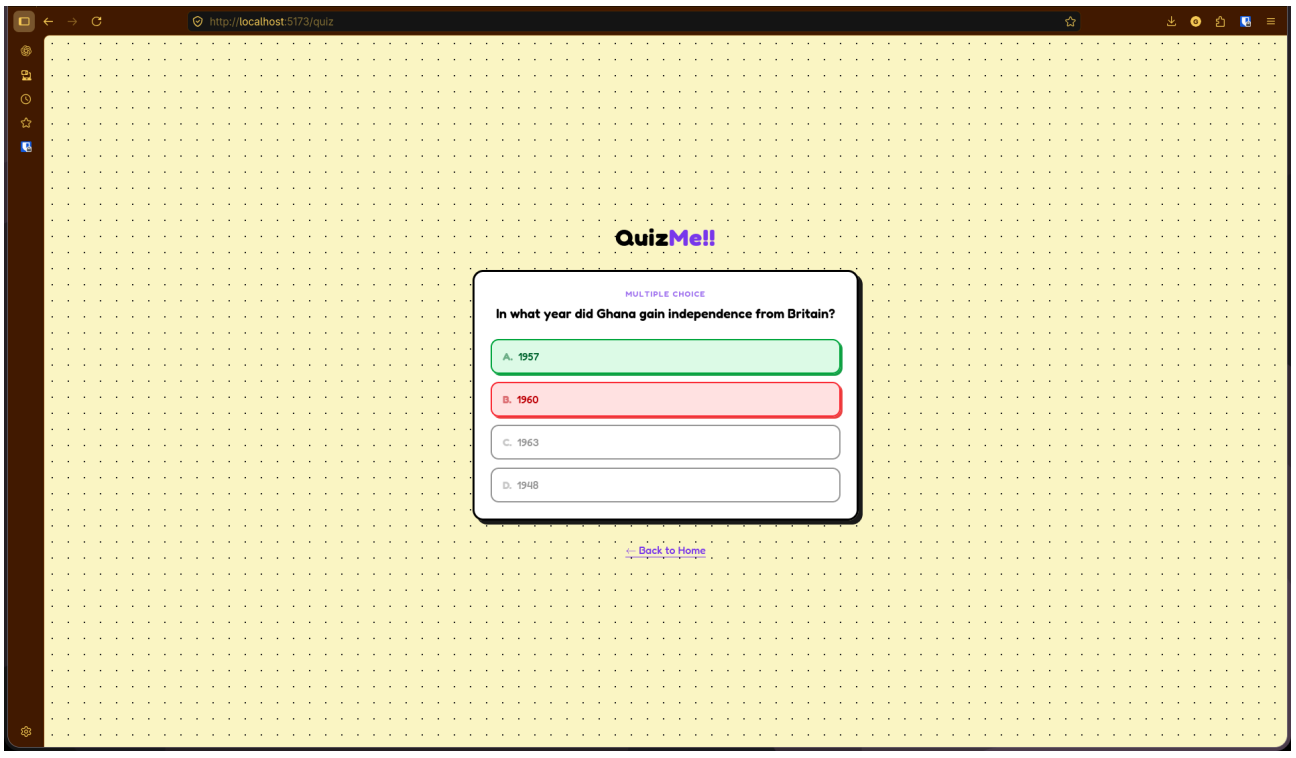
Story	Points
1. Quick Start	SP-1
2. Multiple Choice / True-False Questions	SP-3
3. Instant Feedback	SP-3

Also delivered as part of Sprint 1: React + Vite scaffold, Tailwind CSS, Vitest + Testing Library setup, GitHub Actions CI pipeline, Docker/Docker Compose setup, and a passing production build.

See sprint-docs/002_sprint_1.md for the full task breakdown.




http://localhost:5173/quiz
QuizMe!!
MULTIPLE CHOICE
Which is the highest mountain in Africa?
A. Mount Kenya
B. Mount Kilimanjaro
C. Ras Dashen
D. Mount Meru
Submit
← Back to Home

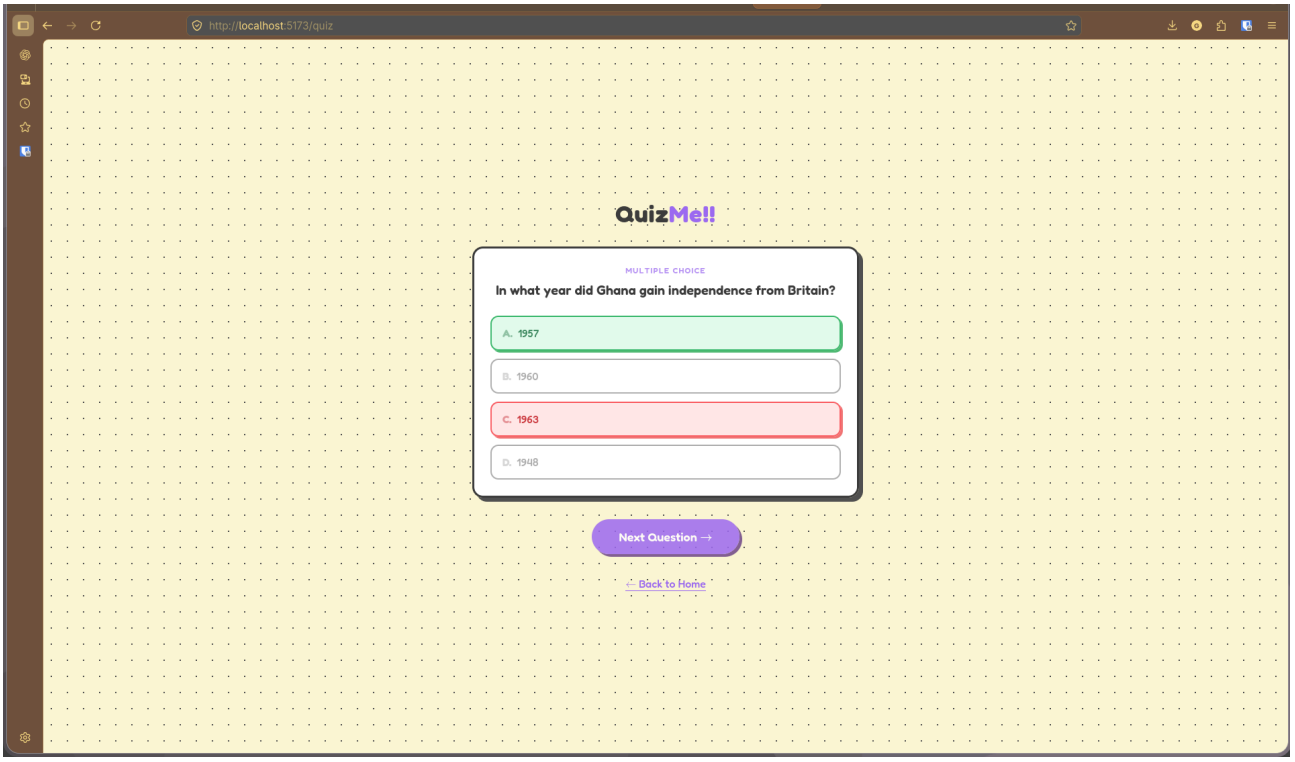

http://localhost:5173/quiz
QuizMe!!
MULTIPLE CHOICE
In what year did Ghana gain independence from Britain?
A. 1957
B. 1960
C. 1963
D. 1948
← Back to Home

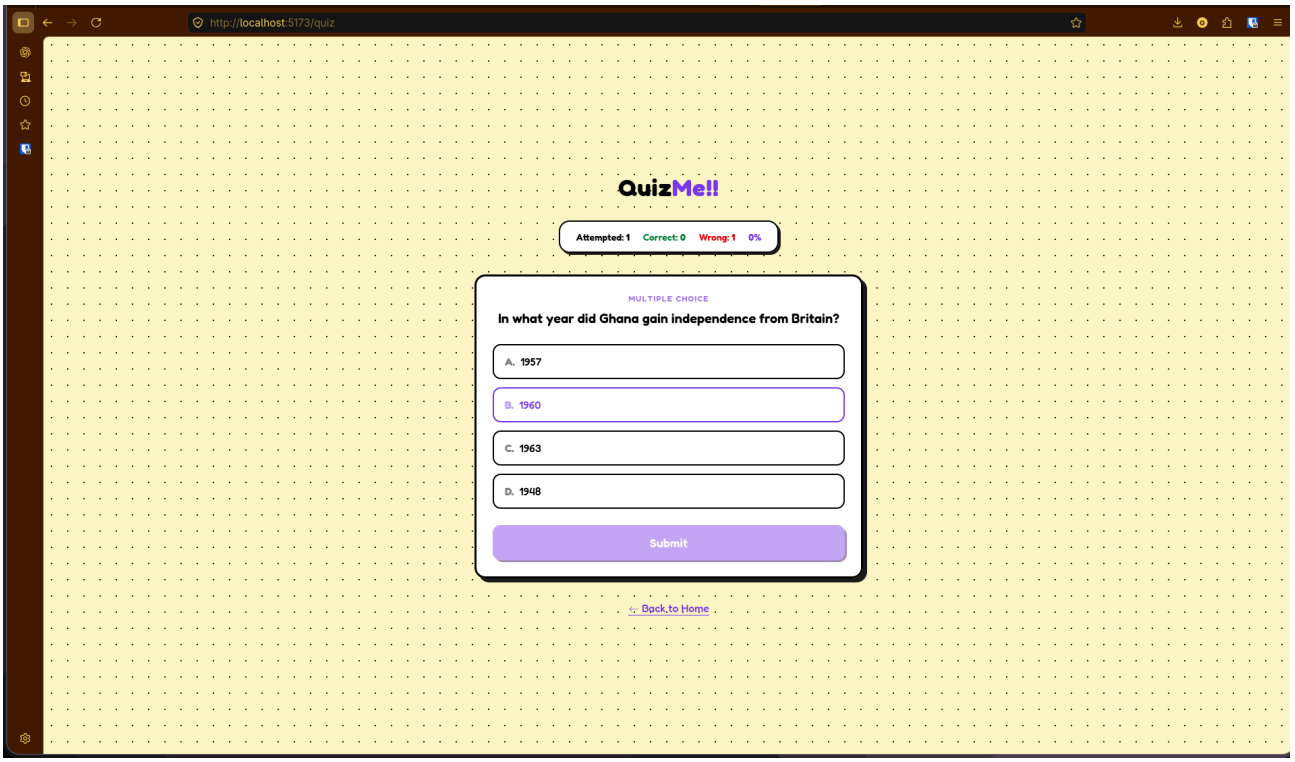
Sprint 2: Navigation, Scoring & Finishing (in progress)

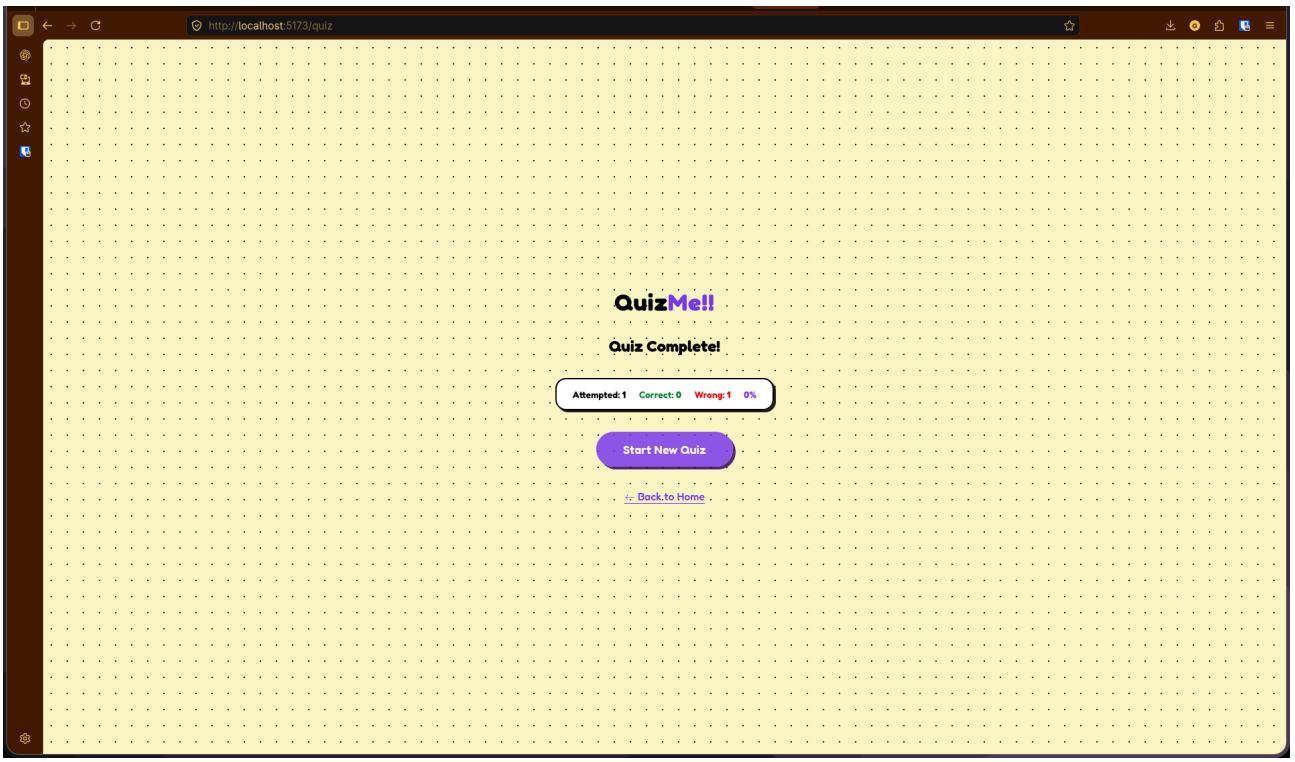
Carried forward from the Sprint 1 retrospective as priorities:

- Story 4 — Next question navigation
- Story 5 — Score tracking (attempted, correct, wrong, percentage)
- Story 7 — Finish quiz (end session, prompt to restart)
- Story 8 — Results screen (general stats + category breakdown)

Stories 4, 5, and 7 have been implemented so far (see commit history below). Story 8 remains open.







Project structure

```
web/
  src/
    components/   Reusable UI pieces (QuestionCard, ScoreBoard, Logo)
    pages/        Route-level pages (Home, Quiz)
```

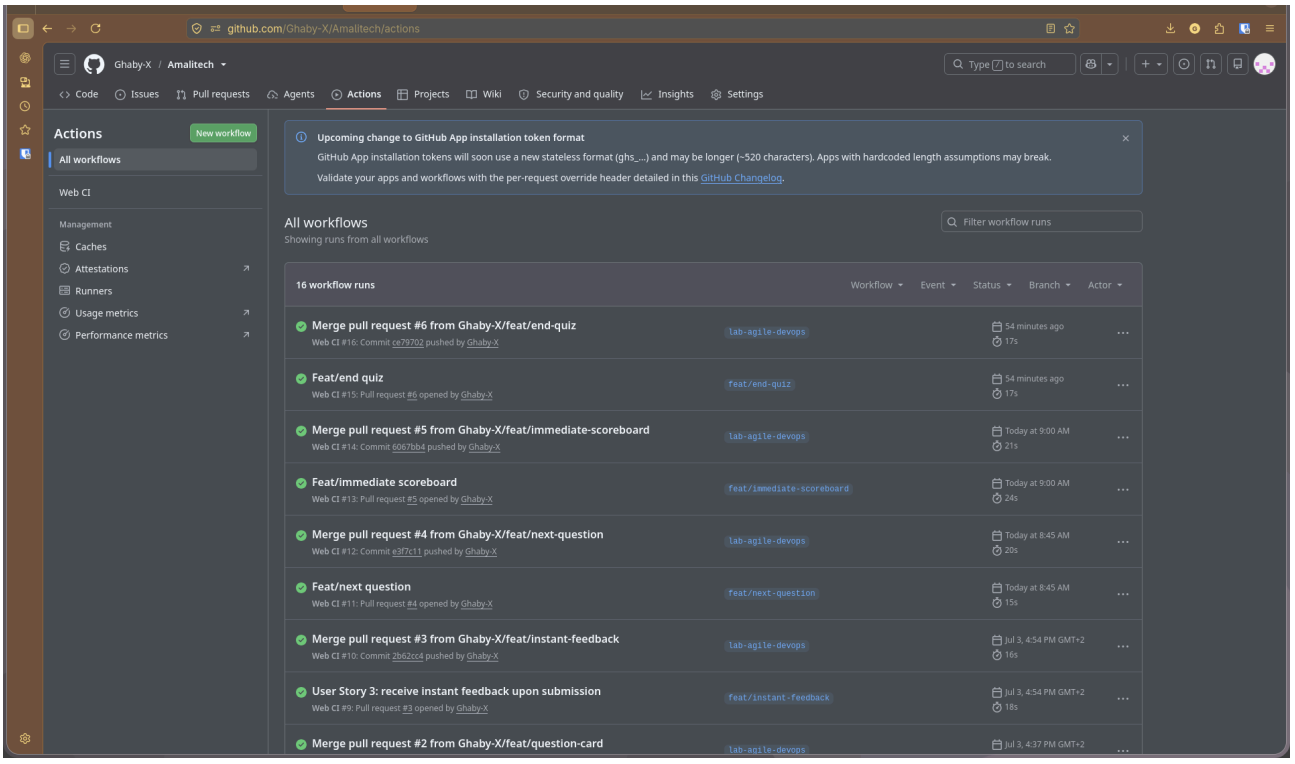
```
  hooks/       Quiz session state management (useQuizSession)
  data/        Question bank and helpers
  test/        Unit tests
Dockerfile

sprint-docs/       Sprint planning, backlog, and retrospectives
docker-compose.yml
```

3. CI/CD Evidence

CI is configured via GitHub Actions at .github/workflows/lab-agile-devops-web-ci.yml:

- Triggers on push and pull request to **lab-agile-devops**, scoped to changes under **agile-devops-lab/web/**.**
- Pipeline runs npm test (Vitest) and npm run build (Vite production build) on every push/PR.
- Deployment/runtime packaging is handled separately via the Dockerfile and docker-compose.yml (multi-stage build -> static assets served by nginx).

Workflow run history and screenshots: see the GitHub repository's Actions tab (link provided at submission).



4. Testing Evidence

Tests are written with Vitest + React Testing Library, colocated in web/src/test/.

Latest local test run

```
$ npm test

 RUN  v4.1.9
```

```
Test Files  7 passed (7)
     Tests  51 passed (51)
```

Test breakdown by file

Test File	Tests	Covers
Logo.test.jsx	3	Logo rendering
Home.test.jsx	4	Home page, Quick Start navigation
QuestionCard.test.jsx	15	MC/T-F rendering, selection, submit, feedback colours
questions.test.js	10	Question bank shape, shuffling, correctness checking
useQuizSession.test.js	9	Session lifecycle: start, answer, next, finish, persistence
ScoreBoard.test.jsx	4	Attempted/correct/wrong counts, percentage calculation
Quiz.test.jsx	6	Live score display, Finish Quiz flow, Next Question flow, finished-state screen
Total	**51**	

Test coverage has grown incrementally with each story (32 tests at the end of Sprint 1, 51 as of the current Sprint 2 stories), matching the Definition of Done requirement that every story ship with unit test coverage.

5. Sprint Review Documents

Sprint 1 Review

- **Sprint Goal**: Deliver a working quiz flow; start a quiz, answer MC/T-F questions, get instant feedback.
- **Planned vs delivered**: 7/7 story points delivered.
- **Stories delivered**: Quick Start (SP-1), MC/True-False questions (SP-3), Instant Feedback (SP-3); all acceptance criteria met.
- **DevOps foundation delivered**: React+Vite scaffold, Tailwind CSS v4, Vitest+Testing Library, 32 passing unit tests, GitHub Actions CI, Docker + Docker Compose,

passing production build.

Full write-up: sprint-docs/003_sprint_1_review.md.

6. Retrospectives

Sprint 1 Retrospective

What went well

- All 3 sprint stories delivered; sprint goal fully met
- Strong test coverage from the start (tests written alongside features)
- DevOps foundation (CI, Docker, production build) completed early, freeing Sprint 2 to focus purely on features
- Clean QuestionCard design made it easy to extend for later stories

What didn't go well

- Switching from CSS modules to Tailwind mid-sprint, plus Tailwind v4 compatibility issues, consumed time that could have gone into features
- CI pipeline debugging required several iterations (Node version deprecation, cache path errors, branch checkout behaviour)
- Button styling went through many small iterative adjustments
- No "end of quiz" state was handled reaching the last question left the user in a dead end; a known gap deferred to Sprint 2

Improvements applied in Sprint 2

1. Agree on visual design decisions (colours, spacing) before implementation instead of iterating live.
2. Review each story for edge cases (empty states, boundary conditions) before coding starts.

Full write-up: sprint-docs/004_sprint_1_retrospective.md.

Sprint 2 retrospective

- All 3 committed stories (4, 5, 7) delivered, 6/6 story points, sprint goal fully met
- Session-lifecycle groundwork from Sprint 1 (`useQuizSession`, localStorage persistence) meant Next/Finish just needed wiring up, not new state design
- The `ScoreBoard` component was built once and reused as-is on both the in-progress quiz and the finished-quiz screen, no duplicated scoring logic
- Agreeing on ScoreBoard placement (top, below logo) and content (attempted/correct/wrong/percentage) before writing code, a direct follow-through on the Sprint 1 retro action item, avoided the back-and-forth

iteration seen last sprint.

What didn't go well

- The Finish Quiz button shipped with a call to an undefined handler (a leftover stub) and wasn't caught until manual testing, because no test file existed yet for the Quiz page itself, only its child components were tested.
- The Finish Quiz button was first scoped to appear only after answering the last question; real usage showed the acceptance criterion ("should be visible to end a quiz") meant visible throughout, which required a rework after the first pass shipped)